

Machine Learning para Finanzas

Clase 6

César Núñez Cuevas

`cnunezc@fen.uchile.cl`



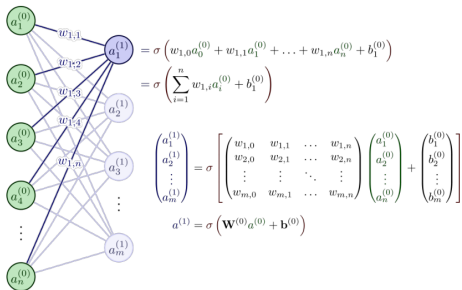
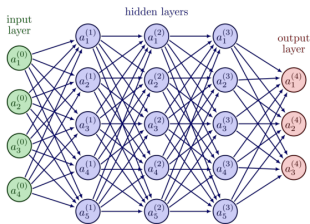
UNIVERSIDAD

Finis Terrae

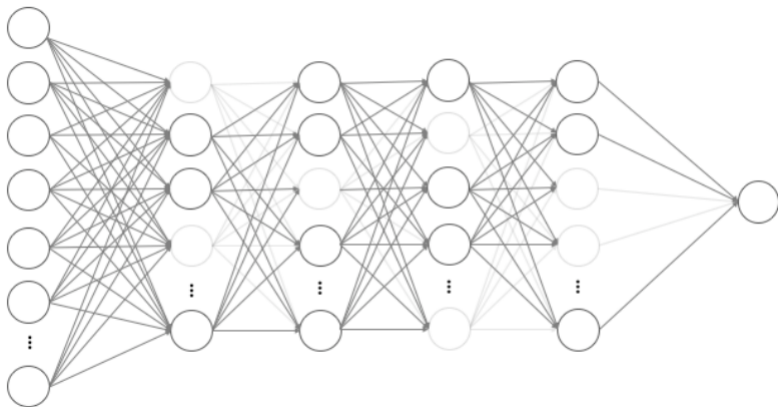
1. Feedforward Neural Networks (FFNN): Arquitectura

- La información fluye en una sola dirección: de la capa de entrada a la salida.
- Estructura: Capa de entrada, n capas ocultas (Hidden Layers) y una capa de salida.
- Cada neurona en la capa L está conectada a todas las neuronas de la capa $L + 1$ (Dense Layers).
- No posee ciclos ni retroalimentación interna (memoria).

1. Feedforward Neural Networks (FFNN): Arquitectura



Dropout Layer



2. FFNN: Formulación Matemática

Cálculo de activación

Para cada capa l :

$$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = \sigma(z^{[l]})$$

- W : Matriz de pesos (weights).
- b : Vector de sesgo (bias).
- σ : Función de activación (ReLU, Sigmoid, Tanh).
- La pérdida (Loss) se calcula al final mediante funciones como MSE o Cross-Entropy.

3. Diferencia Crítica: FFNN vs. MLP

- **MLP (Multi-Layer Perceptron):** Es un subconjunto específico de FFNN. Requiere al menos una capa oculta y utiliza funciones de activación no lineales.
- **FFNN:** Es el concepto genérico. Un Perceptrón simple es una FFNN, pero no un MLP.
- **En Finanzas:** Usamos MLP cuando los datos son tabulares y estáticos (ej: ratios financieros de un balance en un punto en el tiempo).

4. Caso Práctico FFNN: Credit Scoring

- **Entrada:** Datos demográficos, ingresos, historial crediticio, nivel de deuda.
- **Proceso:** La red aprende interacciones no lineales entre el ingreso y la estabilidad laboral.
- **Salida:** Probabilidad de incumplimiento (Default).
- **Ventaja sobre Regresión Logística:** Captura efectos de umbral y saturación de manera automática.

5. Convolutional Neural Networks (CNN)

- Diseñadas originalmente para datos con topología de rejilla (imágenes).
- **Estructura:** Capas de convolución, capas de pooling y capas densas al final.
- **Operación clave:** El kernel o filtro que se desplaza (stride) sobre los datos.
- Extraen características jerárquicas (bordes -¿ formas -¿ objetos).

5. Convolutional Neural Networks (CNN)

Filters - Convolutional Layer

$$\begin{pmatrix}
 0 & 1 & 1 & 1 & 0 & 0 & 0 \\
 0 & 0 & 1 & 1 & 1 & 0 & 0 \\
 0 & 0 & 0 & 1 & 1 & 1 & 0 \\
 0 & 0 & 0 & 1 & 1 & 0 & 0 \\
 0 & 0 & 1 & 1 & 0 & 0 & 0 \\
 0 & 1 & 1 & 0 & 0 & 0 & 0 \\
 1 & 1 & 0 & 0 & 0 & 0 & 0
 \end{pmatrix}
 \begin{matrix}
 \times 1 \times 0 \times 1 \\
 \times 1 \times 0 \times 1 \\
 \times 0 \times 1 \times 0 \\
 \times 1 \times 0 \times 1
 \end{matrix}
 *
 \begin{pmatrix}
 1 & 0 & 1 \\
 0 & 1 & 0 \\
 1 & 0 & 1
 \end{pmatrix}
 =
 \begin{pmatrix}
 1 & 4 & 3 & 4 & 1 \\
 1 & 2 & 4 & 3 & 3 \\
 1 & 2 & 3 & 4 & 1 \\
 1 & 3 & 3 & 1 & 1 \\
 3 & 3 & 1 & 1 & 0
 \end{pmatrix}
 \begin{matrix}
 I \\
 K \\
 I * K
 \end{matrix}$$

Pooling Layer

Input

4	7	1	2
6	3	0	8
9	1	6	0
6	4	1	7

Output

7	8
9	7

Max
pooling

6. CNN: La Operación de Convolución

$$(f * g)(t) = \sum_{\tau} f(\tau)g(t - \tau)$$

- En finanzas, aplicamos **1D-CNN** sobre series temporales.
- El filtro actúa como un detector de patrones técnicos (ej: hombro-cabeza-hombro, canales).
- **Invarianza espacial:** Si un patrón ocurre el lunes o el viernes, la CNN lo detecta igual.

7. CNN: Capas de Pooling (Max y Average)

- **Objetivo:** Reducir la dimensionalidad y controlar el sobreajuste (overfitting).
- **Max Pooling:** Selecciona el valor máximo de una ventana. Mantiene la característica más fuerte detectada.
- **Finanzas:** Ayuda a filtrar el ruido de alta frecuencia en los ticks de precios, manteniendo la señal de tendencia principal.

8. Diferencia: CNN vs. MLP

- **Conectividad Local:** En CNN, una neurona solo se conecta a una región local de la entrada (campo receptivo). MLP es conectividad global.
- **Pesos Compartidos:** El mismo filtro se usa en toda la entrada en CNN. En MLP, cada conexión tiene su propio peso único.
- **Parámetros:** Las CNN tienen muchísimos menos parámetros que un MLP para la misma cantidad de datos de entrada.

9. Caso Práctico CNN: Reconocimiento de Patrones

- **Datos:** Gráficos OHLC (Open, High, Low, Close) convertidos a imágenes o matrices 2D.
- **Uso:** Clasificación automática de patrones de velas japonesas.
- **Efecto:** Elimina la subjetividad del analista técnico humano.
- **Resultado:** Señales de compra/venta basadas en la morfología del precio.

10. Caso Práctico CNN: Datos Alternativos

- **Entrada:** Imágenes satelitales de estacionamientos de centros comerciales o cultivos.
- **Proceso:** CNN cuenta autos o evalúa el verdor del cultivo.
- **Impacto Financiero:** Predicción de ingresos trimestrales de minoristas (Retail) o precios de futuros agrícolas antes del reporte oficial.

11. Recurrent Neural Networks (RNN)

- Diseñadas para datos secuenciales donde el orden importa.
- Poseen conexiones recurrentes que permiten que la información persista.
- **Estado Oculto (h_t):** Funciona como una memoria de corto plazo de los pasos anteriores.

12. RNN: Ecuación del Estado Oculto

$$h_t = \sigma(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

- x_t : Entrada en el tiempo t .
- h_{t-1} : Estado oculto del paso anterior.
- El mismo conjunto de pesos (W_{hh} , W_{xh}) se aplica en cada paso de tiempo.

13. El Problema del Gradiente Desvaneciente

- Al entrenar secuencias largas mediante BPTT (Backpropagation Through Time), los gradientes se multiplican repetidamente.
- **Vanishing Gradient:** El gradiente tiende a cero; la red olvida las dependencias lejanas.
- **Exploding Gradient:** El gradiente crece exponencialmente; la red se vuelve inestable.
- **Finanzas:** Esto impide que una RNN simple aprenda ciclos económicos de largo plazo.

14. Diferencia: RNN vs. MLP

- **Independencia de datos:** MLP asume que x_i es independiente de x_{i-1} . RNN asume dependencia temporal explícita.
- **Entrada Variable:** RNN puede procesar secuencias de longitud variable. MLP tiene una capa de entrada fija.
- **Memoria:** RNN tiene estado interno; MLP es estático.

15. Caso Práctico RNN: Predicción de Retornos

- **Datos:** Retornos diarios de un índice bursátil.
- **Arquitectura:** Many-to-One (Muchos pasos de tiempo para predecir el siguiente).
- **Limitación:** Muy útil para volatilidad de corto plazo, pero falla en capturar estacionalidad anual debido al gradiente desvaneciente.

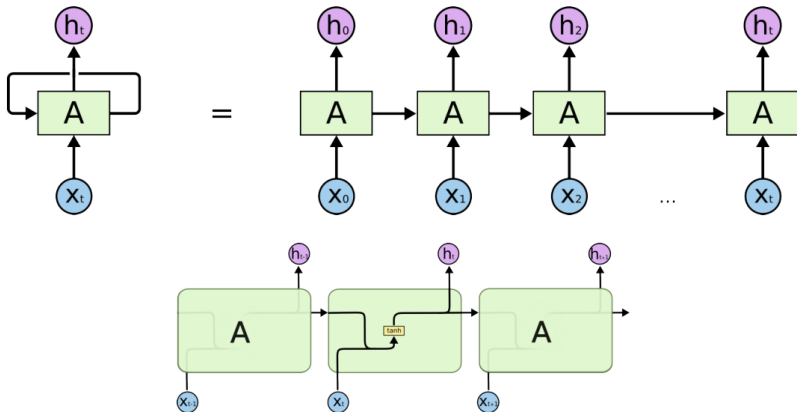
16. Long Short-Term Memory (LSTM)

- Variante de RNN diseñada para solucionar el gradiente desvaneciente.
- Introducida por Hochreiter y Schmidhuber (1997).
- **Componente clave:** La celda de estado (Cell State) que actúa como una cinta transportadora de información a largo plazo.

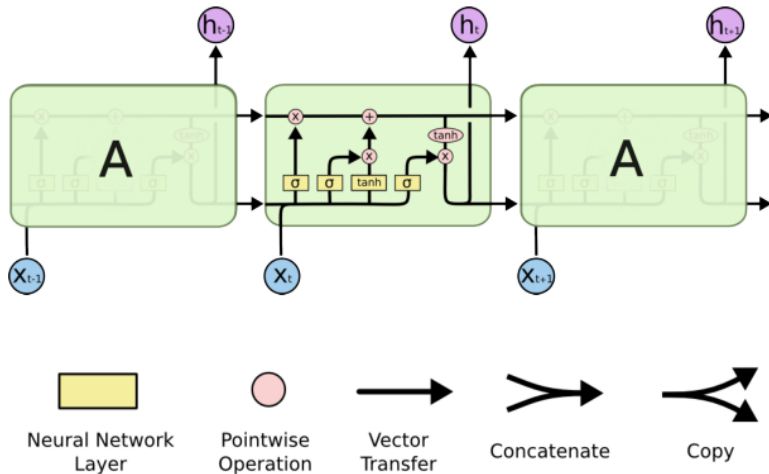
17. LSTM: Las Tres Compuertas (Gates)

- **Forget Gate (f_t):** Decide qué información del estado de la celda se descarta.
- **Input Gate (i_t):** Decide qué nueva información se almacena en el estado de la celda.
- **Output Gate (o_t):** Decide qué parte del estado de la celda se enviará al estado oculto h_t .

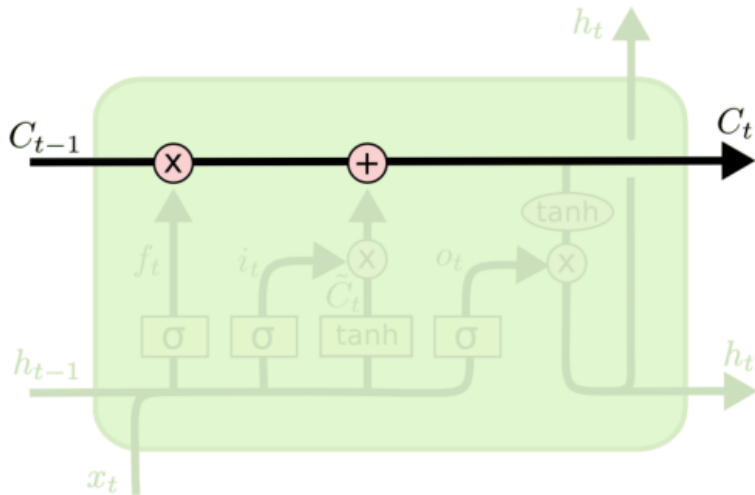
17. LSTM: Las Tres Compuertas (Gates)



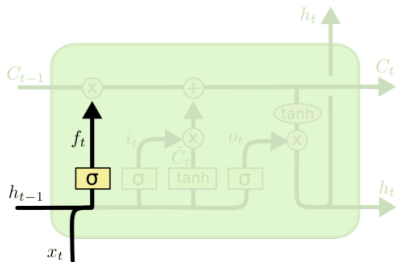
17. LSTM: Las Tres Compuertas (Gates)



17. LSTM: Las Tres Compuertas (Gates)

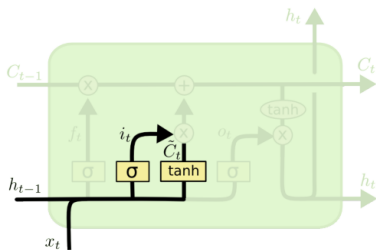


17. LSTM: Las Tres Compuertas (Gates)



$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

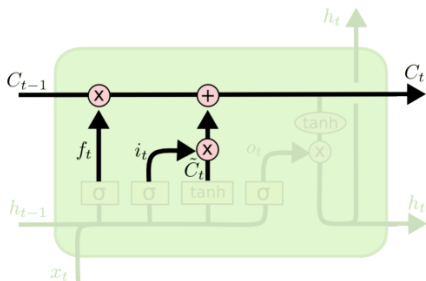
17. LSTM: Las Tres Compuertas (Gates)



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

17. LSTM: Las Tres Compuertas (Gates)



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

18. Matemáticas de la LSTM

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

19. Diferencia: LSTM vs. RNN

- **Memoria:** RNN tiene una memoria "simplez frágil". LSTM tiene una gestión de memoria selectiva mediante compuertas.
- **Estabilidad:** LSTM mantiene gradientes mucho más estables en el tiempo.
- **Costo Computacional:** LSTM tiene 4 veces más parámetros que una RNN por cada neurona oculta.

20. Diferencia: LSTM vs. MLP

- **Contexto:** LSTM entiende el contexto (ej: una caída de precio después de una subida prolongada). MLP solo ve el precio actual.
- **Estructura de pesos:** LSTM utiliza celdas de memoria; MLP solo neuronas de producto punto.

21. Caso Práctico LSTM: Trading Algorítmico

- **Problema:** Predicción de precios en alta frecuencia (HFT).
- **Datos:** Limit Order Book (LOB) con profundidad de mercado.
- **Proceso:** LSTM aprende la dinámica de la liquidez y las órdenes que preceden a un movimiento de precio.
- **Resultado:** Ejecución de órdenes con menor deslizamiento (slippage).

22. Caso Práctico LSTM: Gestión de Riesgos (VaR)

- **Uso:** Estimación del Value at Risk (VaR) no lineal.
- **Ventaja:** A diferencia de modelos GARCH, las LSTM no asumen una distribución de probabilidad específica.
- **Proceso:** La red captura dependencias de largo plazo en la volatilidad (clústeres de volatilidad) de manera más eficiente.

23. Caso Práctico: NLP Financiero

- **Entrada:** Titulares de Bloomberg, minutas de la FED o Tweets de analistas.
- **Arquitectura:** Word Embeddings + LSTM.
- **Objetivo:** Análisis de sentimiento y su impacto inmediato en el precio de los activos.
- **Por qué LSTM:** El orden de las palabras en una frase financiera cambia totalmente el sentido (ej: "Rates will not be raised" vs "Rates will be raised").

24. Tabla Comparativa vs. MLP

Modelo	Estructura Clave	Diferencia vs MLP	Uso Top en Finanzas
CNN	Filtros/Convolución	Localidad espacial/Invarianza	Reconocimiento de Patrones
RNN	Bucles/Estado oculto	Memoria secuencial	Series de tiempo cortas
LSTM	Compuertas/Cell State	Memoria largo plazo selectiva	Predicción de Precios/HFT
MLP	Capas Densas	Estático/Conectividad Total	Credit Scoring / Tabulares

25. Implementación y Stack Tecnológico

- **Librerías:** TensorFlow, PyTorch y Keras para el modelado.
- **Hardware:** Uso masivo de GPUs (NVIDIA) para el entrenamiento paralelo, especialmente en CNN y LSTM.
- **Pipeline:**
 - 1 Ingesta de datos (Pandas/SQL).
 - 2 Normalización (MinMaxScaler/StandardScaler).
 - 3 Entrenamiento con Early Stopping.
 - 4 Backtesting en Out-of-Sample.